

ECE-GY 6143: Introduction to Machine Learning

Midterm, Spring 2019

Prof. Sundeep Rangan

Answer ALL questions. Exam is closed book. No electronic aids. But, you are permitted two cheat sheets, two sides each sheet. Any format for the cheat sheet is permitted. Part marks are given. If you do not remember a particular python command or its syntax, use pseudo-code and state what syntax you are assuming.

You may answer on the exam sheet or in the provided blue book. But, if you use a blue book, please indicate on the exam paper. You must turn in the exam paper at the end of the exam. You may keep your cheat sheet.

Best of luck!

1. *Least squares with a transformation.* Consider the following model for a scalar target $\hat{y}$ from features $\mathbf{x} = (x_1, x_2)$,

$$\hat{y} = f(x, \theta) = \begin{cases} \theta_0 + \theta_1(x_1 + \theta_2 x_2) & \text{if } x_1 < x_2 \\ \theta_0 + \theta_1(1 + \theta_2)x_1 & \text{if } x_1 \geq x_2 \end{cases}$$

where $\theta = (\theta_0, \theta_1, \theta_2)$ are parameters.

- (a) Write this as a linear model. That is, find basis functions $\phi_i(\mathbf{x})$ and parameters β_i such that

$$\hat{y} = \sum_{i=1}^p \beta_i \phi_i(\mathbf{x}).$$

Use a minimum number p of basis functions. Write the linear model parameters β_i in terms of θ_j .

- (b) Given a linear parameter estimate $\hat{\beta}$, how do you find $\hat{\theta}$ in the original model?

2. *Computing the bias.* Suppose that a true function is

$$y = f_0(x) := \min\{2x, 6\}.$$

We get data (x_i, y_i) , $i = 1, \dots, n$ with $y_i = f_0(x_i)$, and fit a model,

$$\hat{y} = f(\mathbf{x}, \hat{\theta}) = \hat{\theta}x, \quad \hat{\theta} = \frac{1}{n} \sum_{i=1}^n \frac{y_i}{x_i}.$$

For each set of the following sets of training data points $\mathbf{x} = (x_1, \dots, x_n)$ and test data point x_{ts} find the bias:

$$\text{Bias}(x_{\text{ts}}) = f(x_{\text{ts}}, \hat{\theta}) - f_0(x_{\text{ts}}).$$

- (a) Training data is $\mathbf{x} = \{1, 2\}$ and test data point is $x_{\text{ts}} = 2$.
- (b) Training data is $\mathbf{x} = \{1, 2\}$ and test data point is $x_{\text{ts}} = 4$.
- (c) Training data is $\mathbf{x} = \{1, 4\}$ and test data point is $x_{\text{ts}} = 4$.

3. *Model selection.* You are given python arrays $\mathbf{x}$ and $\mathbf{y}$ where $\mathbf{x}$ is an $n \times p$ data matrix and $\mathbf{y}$ is a n dimensional vector of continuous targets. For each model order $d=0,1,\dots,p$, you consider a linear regression model using only the first d features:

$$\hat{y}[i] = b + \mathbf{x}[i,0]*\mathbf{w}[0] + \dots + \mathbf{x}[i,d-1]*\mathbf{w}[d-1],$$

where b and $\mathbf{w}[0],\dots,\mathbf{w}[d-1]$ are the parameters. Write python code to:

- Split the data into training and test with approximately 50% of the samples in each.
- Use simple cross-validation to find the optimal order d .

You do not need to use K -fold validation or shuffle the data. You can use the normal rule, not the one SE rule. You may use the functions:

```
regr = LinearRegression()    # Constructs a linear regression object
regr.fit(Z,y)                # Fits a model with features Z and outputs y
yhat = regr.predict(Z)       # Predicts outputs given features Z
```

4. *Linear Classification.* You are given the following five training data points, (x_i, y_i) , with binary class labels $y_i \in \{0, 1\}$:

x_i	0	1.5	1.6	3	4
y_i	0	1	1	0	0

Consider a classifier of the form,

$$\hat{y}_i = \begin{cases} 1 & \text{if } z_i > 0 \\ 0 & \text{if } z_i < 0, \end{cases} \quad (1)$$

where z_i is some function of x_i .

- Find β_0, β_1 such that with $z_i = \beta_0 + \beta_1 x_i$, the classifier (1) makes a minimum number of errors. Indicate which training data points, if any, are misclassified.
- Find $\beta_0, \beta_1, \beta_2$ such that when $z_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2$, the classifier (1) makes a minimum number of errors. Indicate which training data points, if any, are misclassified.

5. *Regularized least squares.* We are given data (x_i, y_i) , $i = 0, \dots, T-1$ and wish to fit a model of the form,

$$y_i \approx \hat{y}_i = \sum_{j=0}^{d-1} w_j x_{i-j},$$

for some parameters w_j . You can assume $x_j = 0$ for $j < 0$. You decide to use regularized least squares,

$$\hat{\mathbf{w}} := \arg \min_{\mathbf{w}} \frac{1}{2} \|\mathbf{y} - \mathbf{A}\mathbf{w}\|^2 + \lambda \phi(\mathbf{w}).$$

- (a) What are the components A_{ij} of the matrix $\mathbf{A}$?
- (b) Suggest a function $\phi(\mathbf{w})$ if it is known that we should have $w_j = 0$ for most j .
- (c) Suggest a function $\phi(\mathbf{w})$ if it is known that we should have $w_j \geq 0$ and $w_{j+1} \leq w_j$ for most indices j .

For parts (b) and (c), there is no single correct answer.

6. *Computing gradients.* Consider the function,

$$J = \sum_{i=1}^N \ln(1 + e^{z_i}) - z_i y_i, \quad z_i = \sum_{j=1}^p \frac{a_j}{1 + (x_i - b_j)^2}.$$

- (a) Compute the gradient components, $\partial J / \partial a_j$ and $\partial J / \partial b_j$.
- (b) Write a python function to that returns function J and the gradients,

```
def Jeval(...):  
    ...  
    return J, Jgrad_a, Jgrad_b
```

You must determine the arguments for the function. For full credit, avoid for loops.

7. *Gradient descent with a stopping condition.* You are given a python function that returns a loss function $f(\mathbf{a}, \mathbf{b})$ and the gradients with respect to vector parameters $\mathbf{a}$ and $\mathbf{b}$:

```
f, fgrad_a, fgrad_b = feval(a,b)
```

Write a few lines of python code that runs gradient descent for some initial conditions `ainit` and `binit` with a fixed step size `step`. The code is suppose to stop when either of the following conditions occurs:

- `max_iter=500` iterations have been done, or
- $\sum_j (\partial f / \partial a_j)^2 \leq (10)^{-8}$ and $\sum_k (\partial f / \partial b_k)^2 \leq (10)^{-8}$.

In python, the format for a `while` loop is:

```
while not done:
    # do something
```